

# 好未来 “资深高级前端研发工程师” 面经

## 第一轮：技术一面（时长：60分钟）

1. `let` 和 `const` 存在变量提升吗？`typeof a` 在 `a` 未声明的情况下会报错吗？为什么？
2. 讲一下 Event Loop 的执行顺序，把微任务和宏任务的场景都加进去，比如 `setTimeout`，`Promise.then`，`async/await`。
3. 写一个 `Promise.all` 的实现。需要考虑哪些边界情况？
4. ES6 的 `Proxy` 和 `Object.defineProperty` 的核心区别是什么？它们在实现数据响应式时各有什么优缺点？
5. Vue 3 的 Composition API 相比于 Vue 2 的 Options API，你认为主要解决了什么问题？在实际项目中，你是如何选择使用哪一种的？
6. 看过 Vue 或者 React 的源码吗？简单说一下 Vue 的模板编译（template compile）过程大概是怎样的。
7. 讲一下 Virtual DOM 的 Diff 算法。当一个列表中，你把最后一个元素移动到最前面，Diff 过程是怎样的？key 在这里起到了什么作用？
8. Webpack 和 Vite 的核心工作原理有什么不同？Vite 在开发环境下为什么会比 Webpack 快？它的预构建（pre-bundling）解决了什么问题？
9. 从输入 URL 到页面完全加载，整个流程是怎样的？尽可能说得详细一些，包括网络、浏览器渲染等各个环节。
10. CSS 中，`position` 的 `absolute` 和 `fixed` 有什么区别？`em`，`rem`，`px` 这些单位又有什么区别和使用场景？
11. 实现一个函数，可以千分位格式化一个数字。例如，输入 `1234567.89`，输出 `'1,234,567.89'`。

## 第二轮：技术二面（时长：60-70分钟）

1. 在XX项目里，你主要负责哪一块？遇到了什么技术难题？你是怎么分析和解决的？
2. 做过性能优化吗？具体从哪些方面入手的？你衡量优化效果的指标是什么？有没有一套完整的监控和度量体系？
3. 讲讲浏览器的缓存策略。强缓存和协商缓存具体的实现方式是怎样的？分别用在什么场景下？
4. 在做首屏加载优化时，你都用过哪些具体的方法？比如 Code Splitting, Tree Shaking, 图片懒加载等，它们的原理是什么？
5. 前端安全问题你了解多少？XSS 和 CSRF 的攻击原理和防御措施分别是什么？

6. 如果现在有一个需求，要做一个在线协作的文档编辑器，类似腾讯文档或者 Google Docs，从前端的角度，你会如何进行技术选型和架构设计？需要考虑哪些核心的技术难点？（比如多人实时协同、冲突处理、大数据量渲染等）
7. 你们团队的 CI/CD 流程是怎样的？从代码提交到最终上线，都经过了哪些环节？你在其中扮演什么角色？
8. Node.js 你熟悉吗？用它做过什么？它和浏览器环境下的 JavaScript 有什么核心区别？
9. 对于 TypeScript，你的看法是什么？它解决了什么问题？带来了哪些新的问题？在你的项目中，类型定义是怎么管理的？
10. 你如何保证你的代码质量？除了 Code Review 和单元测试，还有其它方法吗？

### 第三轮：技术三面（时长：45-60分钟）

1. 看你简历上写负责过好几个项目，你觉得哪个项目对你的成长最大？为什么？
2. 当你的技术方案和团队其他成员，甚至你的领导产生分歧时，你会怎么处理？举一个实际的例子。
3. 如何带领一个小组（比如3-5个前端）完成一个比较复杂的项目？你会如何做技术选型、拆分任务、控制进度和风险？
4. 你认为一个“资深高级”前端工程师，应该具备哪些区别于普通高级工程师的核心能力？
5. 你是如何保持自己的技术视野和竞争力的？最近在关注哪些新的前端技术或者趋势？你对它们有什么看法？
6. 如果我们现在有一个历史项目，技术栈很旧，维护成本很高，性能也很差。如果交给你来负责，你的改造思路是怎样的？会从哪些方面着手？
7. 从你的角度来看，前端工程师的价值体现在哪里？是交付页面，还是其他什么？
8. 你在工作中，有没有主动发现现有系统的问题，并推动解决，最终拿到结果的例子？讲一个。
9. 你过往的工作经历中，有没有觉得比较失败或者遗憾的事情？你从中复盘和学到了什么？
10. 你对我们公司或者我们做的业务有什么了解吗？
11. 你现在看机会，主要考虑哪些因素？

### 一面参考答案：

1. `let` 和 `const` 存在变量提升吗？`typeof a` 在 `a` 未声明的情况下会报错吗？为什么？

`let` 和 `const` 不存在变量提升 (Hoisting)。

更准确地说，它们存在“暂时性死区” (Temporal Dead Zone, TDZ)。虽然 `let` 和 `const` 声明的变量在作用域顶部也会被创建，但在代码执行到声明语句之前，这些变量是无法被访问的。试图在声明前访问它们，会抛出 `ReferenceError` 错误。

- **变量提升 (Hoisting)**: 在 JavaScript 中，`var` 声明的变量和函数声明会被“提升”到其所在作用域的顶部。这意味着你可以在声明之前使用它们（对于 `var` 变量，其值为 `undefined`）。
- **暂时性死区 (TDZ)**: `let` 和 `const` 声明的变量，从它们所在的作用域（块级作用域）开始，到声明语句结束，这中间的区域就是 TDZ。在这个区域内访问变量会报错。

代码示例:

代码块

```
1  console.log(a); // undefined - var 存在变量提升，值为 undefined
2  var a = 1;
3
4  // console.log(b); // ReferenceError: Cannot access 'b' before initialization
   // - let 存在 TDZ
5  let b = 2;
6
7  {
8    // TDZ for c starts here
9    // console.log(c); // ReferenceError
10   let c = 3; // TDZ for c ends here
11 }
```

---

`typeof a` 在 `a` 未声明的情况下不会报错，会返回 `"undefined"`。

原因:

这是 `typeof` 操作符的一个特殊设计。ECMAScript 规范明确规定，当 `typeof` 的操作数是一个未声明的标识符 (Undeclared Identifier) 时，它不会抛出 `ReferenceError`，而是直接返回字符串 `"undefined"`。

这个特性在某些场景下很有用，比如检查一个全局变量是否存在，而不用担心因变量不存在而导致程序崩溃。

代码块

```
1  // a 从未被声明过
2  console.log(typeof a); // "undefined"
3
4  let b;
5  console.log(typeof b); // "undefined"
```

```
6
7 // console.log(c); // ReferenceError - 直接访问未声明的变量会报错
```

总结:

| 声明方式  | 变量提升             | 暂时性死区 (TDZ) | typeof 未声明变量 |
|-------|------------------|-------------|--------------|
| var   | ✅ (值为 undefined) | ❌           | "undefined"  |
| let   | ❌                | ✅           | "undefined"  |
| const | ❌                | ✅           | "undefined"  |

## 2. Event Loop 的执行顺序（宏任务与微任务）

Event Loop（事件循环）是 JavaScript 实现异步的核心机制。其执行顺序可以概括为以下步骤：

1. **执行同步代码**：首先，执行调用栈（Call Stack）中的所有同步代码，直到栈为空。
2. **执行微任务**：检查微任务队列（Microtask Queue）。如果队列不为空，则依次执行队列中所有的微任务，直到队列为空。如果在执行微任务的过程中又产生了新的微任务，那么这些新的微任务也会被添加到队列的末尾，并在当前轮次的微任务执行中被执行。
3. **执行一个宏任务**：从宏任务队列（Macrotask Queue，也叫 Task Queue）中取出一个任务来执行。
4. **重复循环**：执行完一个宏任务后，回到第 2 步，再次检查微任务队列，然后继续下一个宏任务。这个过程不断重复，就构成了事件循环。

简单的说：一次事件循环 = 执行一个宏任务 + 执行所有微任务

常见任务分类:

- **宏任务 (Macrotask)**: `script` (整体代码)、`setTimeout`, `setInterval`, `I/O`, `UI rendering`。
- **微任务 (Microtask)**: `Promise.then`, `Promise.catch`, `Promise.finally`, `async/await` (其 `await` 后面的代码可以看作是 `then` 的内容), `process.nextTick` (Node.js 环境)。

场景示例:

代码块

```

1  console.log('1. script start'); // 同步任务
2
3  setTimeout(function() {
4      console.log('2. setTimeout'); // 宏任务
5  }, 0);
6
7  Promise.resolve().then(function() {
8      console.log('3. promise1'); // 微任务
9      Promise.resolve().then(function() {
10         console.log('4. promise2'); // 微任务
11     });
12 }).then(function() {
13     console.log('5. promise3'); // 微任务
14 });
15
16 async function async1() {
17     console.log('6. async1 start'); // async函数体内的代码是同步的
18     await async2();
19     console.log('7. async1 end'); // await后面的代码是微任务
20 }
21
22 async function async2() {
23     console.log('8. async2'); // 同步
24 }
25
26 async1();
27
28 console.log('9. script end'); // 同步任务

```

## 执行顺序分析:

### 1. 同步代码执行:

- `console.log('1. script start')` -> 输出: `1. script start`
- `setTimeout` 被调用, 其回调函数被放入宏任务队列。
- `Promise.resolve().then(...)` 被调用, 第一个 `.then` 的回调被放入微任务队列。
- `async1()` 被调用。
- `console.log('6. async1 start')` -> 输出: `6. async1 start`
- `await async2()` 执行, 首先执行 `async2()`。
- `console.log('8. async2')` -> 输出: `8. async2`
- `await` 会让出 `async1` 函数的执行权, `await` 后面的代码 `console.log('7. async1 end')` 被放入微任务队列。

- `console.log('9. script end')` -> 输出: `9. script end`

## 2. 微任务执行 (同步代码执行完毕, 检查微任务队列):

- 执行第一个 `Promise.then` 的回调。
- `console.log('3. promise1')` -> 输出: `3. promise1`
- 遇到 `Promise.resolve().then(...)`, 其回调 `console.log('4. promise2')` 被放入微任务队列的末尾。
- 第一个 `then` 执行完毕, 返回的 `Promise` 状态变为 `resolved`, 因此它的下一个 `.then` (`console.log('5. promise3')`) 被放入微任务队列末尾。
- 执行 `await` 后面的微任务。
- `console.log('7. async1 end')` -> 输出: `7. async1 end`
- 执行 `promise1` 中产生的微任务。
- `console.log('4. promise2')` -> 输出: `4. promise2`
- 执行 `promise1` 后面链接的微任务。
- `console.log('5. promise3')` -> 输出: `5. promise3`
- 微任务队列清空。

## 3. 宏任务执行 (UI 渲染发生在此之前, 然后取宏任务):

- 从宏任务队列中取出 `setTimeout` 的回调执行。
- `console.log('2. setTimeout')` -> 输出: `2. setTimeout`

### 最终输出顺序:

代码块

```
1  1. script start
2  6. async1 start
3  8. async2
4  9. script end
5  3. promise1
6  7. async1 end
7  4. promise2
8  5. promise3
9  2. setTimeout
```

## 3. `Promise.all` 的实现及边界情况

`Promise.all` 接收一个 Promise 数组（或者任何可迭代对象），并在所有 Promise 都成功（resolved）后，返回一个新的 Promise，该 Promise 的结果是所有 Promise 结果组成的数组。如果其中任何一个 Promise 失败（rejected），则 `Promise.all` 会立即失败，并返回那个失败的 Promise 的原因。

#### 实现思路:

1. 返回一个新的 `Promise`。
2. 遍历输入的 `promises` 数组。
3. 用一个计数器 `count` 记录已成功的 Promise 数量，用一个数组 `results` 存储每个 Promise 的结果。
4. 对每个 `promise` 使用 `.then()`，成功后将结果存入 `results` 对应位置，并使 `count` 加一。
5. 当 `count` 等于 `promises` 数组的长度时，说明所有 Promise 都成功了，此时 `resolve` 新的 `Promise`，并将 `results` 作为结果。
6. 如果任何一个 `promise` 失败，直接 `reject` 新的 `Promise`，并将失败原因传出。

#### 代码实现:

代码块

```
1  function promiseAll(promises) {
2    return new Promise((resolve, reject) => {
3      // 确保输入是可迭代的if (!promises || typeof promises[Symbol.iterator] !==
      'function') {
4        return reject(new TypeError('Argument must be iterable.'));
5      }
6
7      const promisesArray = Array.from(promises);
8      const results = [];
9      let completedCount = 0;
10     const totalPromises = promisesArray.length;
11
12     // 处理空数组的边界情况if (totalPromises === 0) {
13       return resolve([]);
14     }
15
16     promisesArray.forEach((promise, index) => {
17       // 处理非 Promise 值Promise.resolve(promise).then(result => {
18         results[index] = result;
19         completedCount++;
20         if (completedCount === totalPromises) {
21           resolve(results);
```

```
22     }
23     }).catch(error => {
24         // 只要有一个失败，就立即 reject
25         reject(error);
26     });
27 });
28 });
29 }
```

### 需要考虑的边界情况:

1. **空的可迭代对象**: 如果传入一个空数组 `[]`，`Promise.all` 应该立即 resolve 一个空数组。
2. **非可迭代对象**: 如果传入的参数不是一个可迭代对象（如 `null`，`undefined`，`123`），应该抛出一个错误（或者 reject 一个 `TypeError`）。
3. **包含非 Promise 值**: 如果数组中包含非 Promise 的值（如数字、字符串），应该将它们视为已经 resolved 的 Promise，并将其值直接放入结果数组中。`Promise.resolve()` 可以很好地处理这种情况。
4. **Promise 失败**: 只要有一个 Promise 失败，`Promise.all` 就应该立即 reject，并且 reject 的值是第一个失败的 Promise 的原因。
5. **结果顺序**: 返回的结果数组必须与传入的 Promise 数组的顺序保持一致，而不是按照 Promise 完成的顺序。
6. **异步执行**: 即使传入的 Promise 已经 resolved，`Promise.all` 也应该是异步返回结果的，这由 `new Promise` 的机制保证。

---

## 4. `Proxy` 和 `Object.defineProperty` 的核心区别及优缺点

这两者都是实现 JavaScript 数据响应式的核心机制，但它们在原理和能力上有根本区别。



|         |                                                    |                                                        |
|---------|----------------------------------------------------|--------------------------------------------------------|
| 特性      | Object.defineProperty                              | Proxy                                                  |
| 监听对象    | 只能监听 <b>已存在</b> 的属性。                               | 监听的是 <b>整个对象</b> ，而非对象的属性。                             |
| 新增/删除属性 | 无法直接监听对象属性的新增或删除。(Vue 2 通过 \$set, \$delete API 解决) | 可以轻松监听到属性的新增、删除。( handler.set, handler.deleteProperty) |
| 数组      | 无法直接监听数组的索引修改和 .length 属性的修改。(Vue 2 通过重写数组原型方法解决)  | 可以直接监听数组的索引修改、.length 修改等所有操作。                         |
| 性能      | 在对象初始化时需要遍历所有属性进行设置。                               | 在访问对象属性时才进行代理（懒劫持），性能上可能更优，但代理本身有一定开销。                 |
| API 数量  | 提供了 get 和 set 两个主要的劫持方法。                           | 提供了 13 种拦截方法（如 get, set, has, deleteProperty 等），能力更全面。 |
| 兼容性     | 兼容到 IE9。                                           | ES6 特性，不兼容 IE。                                         |

在实现数据响应式时的优缺点:

Object.defineProperty (Vue 2 使用):

- 优点:
  - 兼容性好，能支持到 IE9。
- 缺点:
  - **无法监听新增/删除属性:** 必须使用 `Vue.set` 或 `this.$set` 等特定 API 才能让新增的属性变成响应式。
  - **无法监听数组变化:** 对于数组的 `push` , `pop` , `splice` 等方法，Vue 2 通过重写数组原型上的这些方法来实现监听，但无法直接通过索引修改数组项（如 `arr[0] = newValue`）或修改 `.length` 来触发响应。
  - **初始化开销大:** 需要在组件初始化时递归遍历 data 对象的所有属性，并为每个属性设置 getter/setter。

## Proxy (Vue 3 使用):

- 优点:
    - **功能强大:** 直接代理整个对象，可以监听到任何类型的操作，包括属性的增、删、改、查。
    - **天然支持数组:** 数组的各种操作都能被自然地拦截到，无需特殊处理。
    - **懒劫持:** 只有当访问到对象的深层属性时，才会递归进行代理，而不是在初始化时就完成所有代理，理论上性能更好。
  - 缺点:
    - **兼容性差:** 是 ES6 的新特性，无法 polyfill，不支持 IE 浏览器。这也是 Vue 3 不支持 IE 的主要原因之一。
    - **Proxy** 返回的是一个新对象，如果不是通过代理对象来操作，那么将无法触发响应式。
- 

## 5. Vue 3 Composition API vs Vue 2 Options API

Composition API 主要解决了 Options API 在大型复杂组件中的几个核心问题:

### 1. 代码组织和可维护性:

- **Options API:** 相关联的逻辑（例如，一个功能的 `data`，`methods`，`computed`，`watch`）被迫分散在不同的选项块中。当组件变得复杂时，要理解或修改一个功能，就需要在代码文件的不同部分之间反复横跳，难以维护。
- **Composition API:** 允许我们将同一个逻辑功能相关的所有代码（响应式状态、方法、计算属性等）组织在一起（通常在一个 `setup` 函数或一个独立的 `use` 函数中）。这使得代码更具内聚性，更容易阅读、理解和维护。

### 2. 逻辑复用:

- **Options API:** 主要通过 `Mixins` 来复用逻辑。但 `Mixins` 存在一些问题：
  - **命名冲突:** 多个 `Mixin` 中的属性或方法可能会相互冲突。
  - **数据来源不清晰:** 在组件中使用一个 `data` 属性时，不清楚它到底是来自哪个 `Mixin`，增加了理解成本。
  - **类型推导不友好:** 对于 TypeScript 等类型系统支持不佳。
- **Composition API:** 通过 **组合式函数 (Composables)** 来复用逻辑。一个 `Composable` 就是一个返回响应式状态和方法的普通函数（例如 `useMousePosition`）。这种方式：
  - **来源清晰:** 使用时像调用普通函数一样，返回的值可以任意命名，不会有命名冲突，来源一目了然。

- **类型支持好:** 可以享受完美的 TypeScript 类型推导。
- **更灵活:** 就像组合普通函数一样，非常灵活。

### 3. 更好的 TypeScript 支持:

- Options API 在 `this` 的类型推导上存在一些困难。
- Composition API 大部分是普通的变量和函数，其类型推导非常自然和强大，能提供更好的代码提示和类型检查。

#### 在实际项目中的选择:

- **优先使用 Composition API ( `<script setup>` ):** 对于所有新项目或新组件，强烈推荐使用 Composition API，特别是结合 `<script setup>` 语法糖。它能带来更清晰的代码结构、更强的逻辑复用能力和更好的类型支持。
- **小型、逻辑简单的组件:** 如果一个组件非常简单，只有少量的 `data` 和 `methods`，使用 Options API 也可以，因为它结构固定，对于新手可能更易上手。
- **维护旧项目:** 在维护基于 Vue 2 或仍然使用 Options API 的 Vue 3 老项目时，继续使用 Options API 保持代码风格一致性是合理的。也可以逐步将复杂的组件重构为 Composition API。
- **混合使用:** Vue 3 允许在同一个项目中混合使用两种 API。甚至可以在一个 Options API 组件中通过 `setup()` 选项来使用 Composition API 的能力。

**结论:** Composition API 是 Vue 团队深思熟虑后的演进方向，它在工程化、可维护性和代码组织方面带来了巨大提升，是现代 Vue 开发的首选。

---

## 6. Vue 模板编译 (Template Compile) 过程

Vue 的模板编译是将用户编写的 `template` 字符串或 `.vue` 文件中的模板部分，转换成**渲染函数 (Render Function)** 的过程。这个渲染函数执行时，会返回虚拟 DOM (Virtual DOM) 树。

#### 大致过程可以分为三个阶段:

##### 1. 解析 (Parse):

- **目的:** 将模板字符串解析成**抽象语法树 (AST)**。
- **过程:** 使用解析器 (Parser) 对模板字符串进行词法分析和语法分析。它会识别模板中的标签、属性、指令 ( `v-if` , `v-for` 等)、插值表达式 ( `{{ a }}` ) 等，并根据它们的层级关系构建一个树形结构的数据对象，即 AST。这个 AST 描述了模板的静态结构和动态绑定信息。

##### 2. 转换 (Transform / Optimize):

- **目的:** 对 AST 进行优化和转换。

- **过程:** 遍历 AST，并对节点进行一系列的转换操作。
  - **优化:** 一个重要的优化步骤是**静态标记 (Static Marking)**。编译器会找出 AST 中完全静态的部分（即永远不会改变的节点），并给它们打上标记。这样，在后续的 Diff 过程中，就可以完全跳过对这些静态节点的比对，从而大大提升性能。
  - **转换:** 处理 `v-if`，`v-for`，`v-model` 等指令，将它们转换成 AST 节点上的一些特定属性或结构，为后续生成代码做准备。

### 3. 代码生成 (Code Generation):

- **目的:** 将优化后的 AST 转换成可执行的渲染函数代码字符串。
- **过程:** 代码生成器 (Generator) 会遍历转换后的 AST，根据节点类型、属性、指令等信息，拼接成一个字符串形式的函数代码。这个函数通常是 `render()` 函数。
- 例如，`<div id="app">{{ message }}</div>` 可能会被编译成类似这样的渲染函数：

代码块

```
1  import { openBlock, createElementBlock, toDisplayString } from "vue"export
   function render(_ctx, _cache, $props, $setup, $data, $options) {
2    return (openBlock(), createElementBlock("div", { id: "app" },
        toDisplayString(_ctx.message), 1 /* TEXT */))
3  }
```

- 这个函数在执行时，会调用 Vue 内部的 `createElementBlock` 等函数来创建 VNode (虚拟 DOM 节点)。

**总结:** `Template` -> `AST` -> (优化后的 AST) -> `Render Function`

这个编译过程既可以在构建时（例如使用 Vite 或 Webpack 的 `vue-loader`）完成，也可以在运行时（如果使用的是完整版的 Vue）完成。推荐在构建时完成，因为这样可以省去浏览器端的编译开销，提升应用性能。

## 7. Virtual DOM 的 Diff 算法

Virtual DOM (VNode) 的 Diff 算法是 Vue 和 React 等框架的核心，用于比较新旧两个 VNode 树，找出最小化的变更，然后只对变更的部分进行真实的 DOM 操作，以提高渲染效率。

Diff 算法的核心是**同层比较**。它不会进行跨层级的节点比较，只会比较同一层级的节点。

**Diff 过程:**

## 1. 根节点比较:

- 如果新旧 VNode 的根节点类型不同（例如 `div` 变成了 `p`），则直接销毁旧的 DOM 树，并用新的 VNode 创建全新的 DOM 树。这是开销最大的操作。
- 如果根节点类型相同，则继续比较它们的属性（attributes）和子节点。

## 2. 属性比较:

- 遍历新旧节点的属性，更新或移除变化的属性。

## 3. 子节点比较（核心和复杂之处）:

- 旧节点有子节点，新节点没有:** 卸载并销毁所有旧的子节点 DOM。
- 旧节点没有子节点，新节点有:** 根据新节点的 VNode 创建并插入所有新的子节点 DOM。
- 新旧节点都有子节点:** 这是最复杂的情况，也是 `key` 发挥作用的地方。Vue 和 React 都采用了一些优化策略，通常是**双端比较 (Double-Ended Diff)** 或类似的算法。

### 双端比较算法简述:

它使用四个指针，分别指向新旧子节点列表的头部和尾部：`oldStartIdx`，`oldEndIdx`，`newStartIdx`，`newEndIdx`。

在一个循环中，进行以下四种比较：

- `oldStartVNode` vs `newStartVNode` (头对头)
- `oldEndVNode` vs `newEndVNode` (尾对尾)
- `oldStartVNode` vs `newEndVNode` (旧头对新尾)
- `oldEndVNode` vs `newStartVNode` (旧尾对新头)

如果匹配成功，就更新节点，并移动对应的指针。如果这四种比较都失败了，就会尝试用 `key` 去旧的子节点列表中查找当前 `newStartVNode` 对应的节点，如果找到了就移动它，如果找不到就创建一个新节点。

---

### 场景分析：把最后一个元素移动到最前面

假设旧的列表 VNode 是 `[A, B, C]`，新的列表 VNode 是 `[C, A, B]`。所有节点的 `key` 都保持不变。

没有 `key` 或 `key` 是 `index` 的情况（就地复用策略）：

Diff 算法会认为这是三个节点的“内容”发生了改变，它会执行：

- 比较旧的 `A` 和新的 `C`，发现内容不同，于是将 `A` 对应的 DOM 内容更新为 `C`。
- 比较旧的 `B` 和新的 `A`，发现内容不同，于是将 `B` 对应的 DOM 内容更新为 `A`。
- 比较旧的 `C` 和新的 `B`，发现内容不同，于是将 `C` 对应的 DOM 内容更新为 `B`。

这导致了**三次 DOM 内容更新**，效率非常低，且如果节点包含状态（如输入框内容、组件内部 state），会导致状态错乱。

有唯一的 **key** 的情况 (例如 `key="A"`，`key="B"`，`key="C"`)

Diff 算法通过 **key** 能够识别出节点是同一个，只是位置变了。过程大致如下：

1. **头对头比较**: 旧 `A(key=A)` vs 新 `C(key=C)` -> key 不同，不匹配。
2. **尾对尾比较**: 旧 `C(key=C)` vs 新 `B(key=B)` -> key 不同，不匹配。
3. **旧头对新尾**: 旧 `A(key=A)` vs 新 `B(key=B)` -> key 不同，不匹配。
4. **旧尾对新头**: 旧 `C(key=C)` vs 新 `C(key=C)` -> **匹配成功!**
  - 算法识别出 `C` 节点需要被移动到当前头部（即 `A` 节点之前）。
  - 执行 DOM 操作：`parent.insertBefore(C_DOM, A_DOM)`。
  - 移动指针，现在旧列表待处理的是 `[A, B]`，新列表待处理的是 `[A, B]`。
5. **继续循环**:
  - 头对头比较：旧 `A` vs 新 `A` -> 匹配成功。
  - 尾对尾比较：旧 `B` vs 新 `B` -> 匹配成功。
6. 循环结束。

最终，只需要执行**一次 DOM 移动操作**，没有销毁或创建任何 DOM 元素，也没有不必要的内容更新。

**key** 的作用：

**key** 是 VNode 的唯一标识。它帮助 Diff 算法高效地识别出哪些节点是新增的、删除的、还是仅仅是移动了位置。通过 **key**，算法可以最大程度地复用已有的 DOM 节点，从而最小化 DOM 操作，显著提升性能，并能保持组件的状态正确性。

---

## 8. Webpack 和 Vite 的核心工作原理不同

| 特性       | Webpack                                            | Vite                                                      |
|----------|----------------------------------------------------|-----------------------------------------------------------|
| 核心原理     | 打包器 (Bundler)                                      | 原生 ESM 驱动的构建工具                                            |
| 开发环境     | 启动时打包：分析整个项目的依赖，将所有模块打包成一个或多个 bundle 文件，然后启动开发服务器。 | 按需编译：利用浏览器对原生 ES Module (ESM) 的支持，只在浏览器请求某个模块时才对其进行编译和提供。 |
| 热更新(HMR) | 修改文件后，需要重新打包受影响的模块，然后推送给浏览器。                       | 利用原生 ESM，只需精确地让浏览器重新请求被修改的那个模块，速度极快。                      |
| 生产环境     | 使用自身的打包能力。                                         | 使用 Rollup 进行打包，因为它更轻量且对 ESM 输出更优化。                        |

### Vite 在开发环境下为什么比 Webpack 快？

根本原因在于工作模式的不同。

- **Webpack:** 启动开发服务器前，必须先完成整个项目的依赖分析、编译和打包。对于大型项目，这个过程可能需要数十秒甚至数分钟。这被称为“冷启动”。即使是热更新，也需要重新计算模块依赖关系并打包，速度会随着项目规模增大而变慢。
- **Vite:** 利用了现代浏览器原生支持的 ES Module (`<script type="module">`)。
  - a. **极速冷启动:** 启动开发服务器时，Vite 几乎不做任何事情，只是启动一个服务器。
  - b. **按需编译:** 当浏览器通过 `import` 语句请求一个模块时 (例如 `import App from './App.vue'`)，Vite 的开发服务器会拦截这个请求，**即时编译**被请求的模块 (`App.vue`)，然后将其作为原生 ES Module 返回给浏览器。
  - c. **HMR 极快:** 当你修改一个文件时，Vite 只需要精确地让浏览器重新请求这一个被修改的模块。由于没有打包过程，HMR 的速度不会随项目规模增大而减慢，基本是毫秒级的。

### Vite 的预构建 (pre-bundling) 解决了什么问题？

尽管 Vite 利用原生 ESM，但在启动开发服务器时，它还是会执行一个“预构建”步骤。这个步骤使用 `esbuild`（一个用 Go 编写的极速打包器）来处理。它主要解决了两个问题：

1. **CommonJS 和 UMD 模块的兼容性:** 原生 ESM 无法直接导入 CommonJS 或 UMD 格式的模块。预构建会将这些非 ESM 格式的第三方依赖（例如 `lodash`，`react` 的旧版本）转换为标准的 ESM 格式，这样浏览器就可以正常加载它们了。



2. **性能优化（减少网络请求）**：第三方库通常由许多小的模块文件组成（例如 `lodash-es` 有几百个模块）。如果在开发中直接使用，每个 `import` 都会触发一次浏览器 HTTP 请求，成百上千的请求会严重阻塞页面加载。预构建将这些零散的模块打包成一个或少数几个大的 ESM 文件。这样，在实际代码中 `import { debounce } from 'lodash-es'` 时，浏览器只需要请求一个预构建好的 `lodash-es.js` 文件，大大减少了网络请求数量，提升了页面加载速度。
- 

## 9. 从输入 URL 到页面完全加载的流程

这是一个非常经典且宏大的问题，涵盖了网络、操作系统、浏览器等多个层面。可以大致分为以下几个主要阶段：

### 1. URL 解析与请求

- **用户输入**: 用户在浏览器地址栏输入 URL。
- **URL 解析**: 浏览器解析 URL，判断协议（HTTP/HTTPS）、域名、端口、路径等。
- **HSTS 检查**: 如果网站在 HSTS (HTTP Strict Transport Security) 列表中，浏览器会强制使用 HTTPS 进行请求。
- **DNS 查询**:
  - 浏览器首先检查自身缓存中是否有该域名对应的 IP 地址。
  - 若无，则检查操作系统 hosts 文件。
  - 若无，则向本地 DNS 服务器发起递归查询。
  - 本地 DNS 服务器会依次向根 DNS 服务器、顶级域 (TLD) DNS 服务器、权威 DNS 服务器查询，最终获取到目标服务器的 IP 地址。
- **TCP 连接 (三次握手)**:
  - 客户端向服务器发送一个 SYN 包（同步序列编号）。
  - 服务器收到后，返回一个 SYN-ACK 包（同步确认）。
  - 客户端收到后，再发送一个 ACK 包（确认），连接建立。
- **TLS/SSL 协商 (如果是 HTTPS)**: 在 TCP 连接建立后，进行 TLS 握手，协商加密算法，交换密钥，建立安全的加密通道。

### 2. 浏览器发送 HTTP 请求

- 浏览器构建 HTTP 请求报文，包括请求行（方法、URL、协议版本）、请求头（User-Agent, Accept, Cookie 等）和请求体（如 POST 数据）。
- 请求报文通过 TCP 连接发送给服务器。



### 3. 服务器处理请求并响应

- Web 服务器（如 Nginx, Apache）接收到请求。
- 服务器根据请求的 URL 和参数，可能需要查询数据库、调用后端服务等，最终生成响应数据。
- 服务器构建 HTTP 响应报文，包括状态行（协议版本、状态码如 200 OK）、响应头（Content-Type, Set-Cookie 等）和响应体（通常是 HTML 内容）。
- 服务器将响应报文发送回浏览器。

### 4. 浏览器接收并解析响应

- 浏览器接收到响应报文。
- 如果状态码是 301/302 等重定向，浏览器会根据响应头中的 `Location` 字段重新发起请求。
- 如果状态码是 200，浏览器开始解析响应体。
- **关闭或保持连接:** 根据响应头中的 `Connection: keep-alive` 决定是否保持 TCP 连接，以便后续请求复用。如果是 HTTP/2，则连接会被多路复用。

### 5. 浏览器渲染页面

- **构建 DOM 树:** 浏览器解析 HTML 文档，将标签转换为 DOM 节点，构建成一个 DOM 树。这个过程是自上而下，同步进行的。
- **构建 CSSOM 树:** 在解析 HTML 的过程中，如果遇到 `<link>` 标签或 `<style>` 标签，浏览器会异步下载并解析 CSS 文件，构建 CSSOM (CSS Object Model) 树。CSS 解析不会阻塞 DOM 解析，但会阻塞 DOM 渲染。
- **执行 JavaScript:** 如果遇到 `<script>` 标签（没有 `async` 或 `defer` 属性），浏览器会停止解析 HTML，立即下载并执行 JavaScript。JS 的执行会阻塞 DOM 的构建。这就是为什么通常建议将 `<script>` 标签放在 `<body>` 底部的原因。
  - `defer`: 脚本会异步下载，并等到 HTML 解析完成后、`DOMContentLoaded` 事件触发前执行。
  - `async`: 脚本会异步下载，下载完成后立即执行，可能会阻塞 HTML 解析。
- **构建渲染树 (Render Tree):** 浏览器将 DOM 树和 CSSOM 树结合起来，生成渲染树。渲染树只包含需要被显示的节点（例如 `display: none` 的节点不会在渲染树中）。
- **布局 (Layout / Reflow):** 浏览器根据渲染树，计算出每个节点在屏幕上的确切位置和大小。
- **绘制 (Paint):** 浏览器根据布局计算出的信息，将节点绘制成屏幕上的实际像素。这个过程涉及到将内容分层 (Compositing Layers)，栅格化 (Rasterization) 等。
- **合成 (Composite):** 将绘制好的各个图层按照正确的顺序合成到屏幕上，最终显示页面。

### 6. 页面加载完成

- 所有资源（HTML, CSS, JS, 图片等）都加载完毕后，`window.onload` 事件被触发。
-

## 10. CSS position 的 absolute 和 fixed 区别 & 单位区别

### position: absolute vs position: fixed

|       |                                                                |                                 |
|-------|----------------------------------------------------------------|---------------------------------|
| 特性    | position: absolute (绝对定位)                                      | position: fixed (固定定位)          |
| 定位基准  | 相对于最近的、非 static 定位的祖先元素进行定位。如果没有这样的祖先，则相对于初始包含块（通常是 <html>）定位。 | 相对于浏览器视口 (viewport) 进行定位。       |
| 滚动影响  | 会随着页面的滚动而滚动。                                                   | 不会随着页面的滚动而滚动，始终固定在屏幕的某个位置。      |
| 脱离文档流 | ✅ (元素脱离正常的文档流，不占据空间)                                           | ✅ (元素脱离正常的文档流，不占据空间)            |
| 使用场景  | 对话框、下拉菜单、需要在一个特定容器内定位的元素。                                      | 顶部导航栏、回到顶部按钮、侧边栏广告等需要固定在屏幕上的元素。 |

### em, rem, px 单位的区别和使用场景

| 单位            | 描述                                                                   | 优点                                                                  | 缺点                                                                                      | 使用场景                                                                                                               |
|---------------|----------------------------------------------------------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| px (Pixel)    | <b>绝对单位</b> ，像素。表示屏幕上的一个物理点。                                         | 精确，稳定，易于理解。                                                         | 无法随用户浏览器字体大小设置而缩放，不利于可访问性。                                                              | 需要精确控制大小的场景，如边框 1px、图片尺寸。                                                                                          |
| em            | <b>相对单位</b> ，相对于其父元素的 font-size。如果自身设置了 font-size，则相对于自身的 font-size。 | 可继承，可以创建可伸缩的布局。                                                     | 容易产生 <b>复合效应</b> 。如果多层嵌套的元素都使用 em，计算最终大小时会变得非常复杂和不可预测 (1.2em 的父元素下的 1.2em 最终是 1.44em )。 | 需要根据父元素字体大小进行缩放的局部组件，如按钮的 padding、margin。                                                                          |
| rem (Root em) | <b>相对单位</b> ，相对于根元素 (<html>) 的 font-size。                            | 解决了 em 的复合效应问题，所有元素的尺寸都只与一个基准 (根字体大小) 相关，计算简单，易于维护。可以轻松实现整个页面的整体缩放。 | 需要在 html 元素上设置一个基准 font-size。                                                           | 移动端适配的主流方案。通过 JS 动态修改 html 的 font-size，可以实现页面元素在不同屏幕尺寸下的等比缩放。常用于设置组件的 width, height, font-size, margin, padding 等。 |

总结与建议:

- **主体**: 使用 `rem` 做整体布局和尺寸控制，实现响应式和可访问性。
- **局部**: 在特定组件内部，如果希望内外边距等随组件字体大小变化，可以使用 `em`。
- **点缀**: 对于需要像素级精确的边框或小图标，使用 `px`。

## 11. 实现千分位格式化函数

这个函数需要处理整数、小数、负数和零等情况。

实现思路:

1. 将数字转换为字符串。
2. 使用 `split('.')` 将字符串分为整数部分和小数部分。
3. 对整数部分进行处理:

- 使用正则表达式 `/\B(?=(\d{3})+(?!\d))/g` 是最简洁高效的方法。
  - `\B`：匹配非单词边界，确保不会在数字的开头添加逗号。
  - `(?=(\d{3})+(?!\d))`：这是一个正向先行断言，它匹配一个位置，这个位置后面跟着一组或多组的“三位数字”，并且这组数字后面不能再有数字（即到达字符串末尾）。
- 将匹配到的位置替换为逗号 `,`。

4. 将处理好的整数部分和小数部分（如果存在）用 `.` 拼接起来。

#### 代码实现:

代码块

```
1 function formatThousandth(number) {
2   // 1. 处理非数字或 null/undefined 的情况 if (number === null || number ===
   undefined || isNaN(number)) {
3     return '';
4   }
5
6   // 2. 将数字转为字符串 const numStr = String(number);
7
8   // 3. 分离整数和小数部分 let [integerPart, decimalPart] = numStr.split('.');
9
10  // 4. 使用正则表达式为整数部分添加逗号 // 正则表达式解析: // \B - 匹配一个
   非单词边界，确保我们不在字符串的开头添加逗号。 // (?=...) - 正向先行断言，匹配一个
   位置，但不消耗字符。 // (\d{3})+ - 匹配一组或多组三个数字。 // (?!\d) - 负向
   先行断言，确保这三位数字的后面没有其他数字了（即到达了字符串的末尾或小数点）。 const
   integerRegex = /\B(?=(\d{3})+(?!\d))/g;
11   integerPart = integerPart.replace(integerRegex, ',');
12
13   // 5. 拼接整数和小数部分 return decimalPart !== undefined ?
   `${integerPart}.${decimalPart}` : integerPart;
14 }
15
16 // --- 测试 --- console.log(formatThousandth(1234567.89)); // 输出:
   '1,234,567.89' console.log(formatThousandth(1234567)); // 输出:
   '1,234,567' console.log(formatThousandth(-1234567.89)); // 输出:
   '-1,234,567.89' console.log(formatThousandth(123.45)); // 输出:
   '123.45' console.log(formatThousandth(0)); // 输出:
   '0' console.log(formatThousandth(null)); // 输出:
   '' console.log(formatThousandth('not a number')); // 输出: ''
```

## 二面参考答案:

## 1. 性能优化、衡量指标与监控体系

入手的方面主要分为三大类：

### 1. 加载性能优化 (Loading Performance):

#### ◦ 网络层面:

- **资源压缩与合并:** 使用 Webpack/Vite 等工具对 JS/CSS 进行压缩、混淆，减少文件体积。对于小图标，使用雪碧图 (CSS Sprites) 或 SVG Sprites 减少 HTTP 请求。
- **HTTP/2 或 HTTP/3:** 启用 HTTP/2 的多路复用、头部压缩等特性，减少网络延迟。
- **CDN:** 将静态资源部署到 CDN，利用其边缘节点加速用户访问。
- **DNS 预解析:** 使用 `<link rel="dns-prefetch" href="//example.com">` 提前解析域名。

#### ◦ 资源本身:

- **Code Splitting:** 按需加载代码，将首屏不需要的 JS 分离出去。
- **Tree Shaking:** 移除代码中未被引用的部分，减小 bundle 体积。
- **图片优化:**
  - **图片懒加载 (Lazy Loading):** 对于视口外的图片，延迟加载。
  - **响应式图片:** 使用 `<picture>` 标签或 `srcset` 属性，为不同屏幕尺寸提供最合适的图片。
  - **图片格式:** 使用 WebP 等现代图片格式，压缩率更高。
- **字体优化:** 字体文件按需加载，或只加载需要的字形。

### 2. 渲染性能优化 (Rendering Performance):

#### ◦ 浏览器渲染路径:

- **减少重排 (Reflow) 与重绘 (Repaint):** 避免频繁操作 DOM 样式。批量修改样式（例如通过修改 class），或者使用 `document.fragment`。
- **CSS 优化:** 避免使用复杂的 CSS 选择器。将耗时长动画、变换效果（如 `transform`，`opacity`）通过 `will-change` 或 `translateZ(0)` 提升到合成层 (Compositing Layer)，利用 GPU 加速。

#### ◦ JavaScript 执行:

- **长任务拆分:** 对于耗时长的 JS 计算，使用 `setTimeout` 或 `requestAnimationFrame` 将其分解成小任务，避免阻塞主线程。
- **Web Workers:** 将复杂的计算任务放到 Worker 线程中，不影响 UI 渲染。
- **框架优化:**

- **Vue:** 合理使用 `key`，使用 `v-show` 代替 `v-if` 处理频繁切换的场景，对于大数据列表使用虚拟滚动 (Virtual Scrolling)。
- **React:** 使用 `shouldComponentUpdate`，`React.memo`，`useMemo`，`useCallback` 避免不必要的组件重渲染。

### 3. 用户感知性能优化:

- **骨架屏 (Skeleton Screen):** 在数据加载完成前，显示页面的大致轮廓，提供更好的用户预期。
- **占位符:** 为图片、视频等资源预留空间，避免加载完成后页面抖动。
- **预加载 (Preload) / 预取 (Prefetch):** 使用 `<link rel="preload">` 加载当前页面必须的资源，使用 `<link rel="prefetch">` 加载未来可能用到的资源。

### 衡量优化效果的指标:

我主要关注 Google 提出的 **Web Vitals (核心 Web 指标)** 和其他一些关键指标:

- **LCP (Largest Contentful Paint):** 最大内容绘制。衡量**加载性能**。目标是小于 2.5 秒。
- **FID (First Input Delay):** 首次输入延迟。衡量**交互性**。目标是小于 100 毫秒。
- **CLS (Cumulative Layout Shift):** 累积布局偏移。衡量**视觉稳定性**。目标是小于 0.1。
- **FP (First Paint):** 首次绘制。页面第一个像素出现的时间。
- **FCP (First Contentful Paint):** 首次内容绘制。页面上任何一部分内容（文本、图片等）首次绘制的时间。
- **TTI (Time to Interactive):** 可交互时间。页面达到布局稳定、主要内容可见、并且可以响应用户输入的时间。

### 完整的监控和度量体系:

是的，建立一套体系至关重要，它包括:

#### 1. 数据采集 (RUM - Real User Monitoring):

- 使用 `performance.timing` API 和 `PerformanceObserver` API 采集上述性能指标。
- 封装一个轻量级的 SDK，在用户端收集数据，包括用户的设备、网络环境、地理位置等信息。
- 监控 JS 错误、资源加载失败等异常情况。

#### 2. 数据上报:

- 将采集到的数据在合适的时机（如 `beforeunload` 事件）通过 `navigator.sendBeacon` API 或图片信标 (Image Beacon) 的方式上报到数据分析服务器，避免影响主业务请求。

#### 3. 数据分析与可视化:

- 将上报的数据进行清洗、聚合，存储到数据库中（如 ClickHouse）。
- 搭建一个前端性能监控平台（可以自研或使用 Sentry, Datadog 等第三方服务），通过仪表盘 (Dashboard) 可视化展示各项性能指标的趋势（如 P90, P75 分位值）、分布情况等。

#### 4. 告警与分析:

- 设置性能基线 (Baseline)，当线上性能数据出现大幅波动或劣化时，触发告警。
  - 通过性能数据与业务数据（如转化率、跳出率）的关联分析，量化性能优化带来的业务价值。
  - 定期出具性能报告，驱动团队持续进行性能优化。
- 

#### 5. 浏览器的缓存策略

浏览器的缓存策略是前端性能优化的核心一环，主要分为**强缓存 (Strong Cache)**和**协商缓存 (Negotiation Cache)**两大类。

##### 强缓存:

- **原理:** 浏览器在请求资源时，会先检查本地缓存。如果缓存未过期，则**不向服务器发送任何请求**，直接从本地缓存中读取资源，HTTP 状态码为 `200 (from memory/disk cache)`。
- **实现方式 (HTTP Header):**
  - a. **Expires (HTTP/1.0):**
    - 值为一个绝对时间的 GMT 格式字符串，如 `Expires: Wed, 21 Oct 2026 07:28:00 GMT`。
    - **缺点:** 依赖于客户端本地时间，如果用户本地时间不准确，可能会导致缓存失效或提前过期。
  - b. **Cache-Control (HTTP/1.1):**
    - 值为相对时间，如 `Cache-Control: max-age=31536000` (单位是秒)。它指定了资源在被缓存后的多长时间内有效。
    - **优点:** 解决了 `Expires` 的问题，优先级也高于 `Expires`。
    - **常用指令:**
      - `public`: 资源可以被任何中间代理（如 CDN、代理服务器）缓存。
      - `private`: 只能被最终用户的浏览器缓存。
      - `no-cache`: **强制进行协商缓存**。浏览器会使用缓存，但每次都会向服务器验证资源是否过期。
      - `no-store`: **禁止任何形式的缓存**。浏览器每次都必须重新请求资源。

##### 协商缓存:

- **原理:** 当强缓存失效（或被设置为 `no-cache`），浏览器**必须向服务器发送请求**，来验证本地缓存的资源是否仍然有效。服务器根据请求头中的信息判断。



- 如果资源未改变，服务器返回 `304 Not Modified` 状态码，响应体为空，浏览器直接使用本地缓存。
- 如果资源已改变，服务器返回 `200 OK` 状态码，并附带最新的资源内容，浏览器更新本地缓存。
- **实现方式 (请求头 / 响应头):**
  - a. **Last-Modified / If-Modified-Since :**
    - **首次响应:** 服务器在响应头中加入 `Last-Modified`，表示资源的最后修改时间。
    - **后续请求:** 浏览器在请求头中加入 `If-Modified-Since`，值为上次收到的 `Last-Modified` 的值。
    - **服务器验证:** 服务器比较 `If-Modified-Since` 的时间和资源的实际最后修改时间。
    - **缺点:** 时间戳的精度只能到秒，如果在 1 秒内文件被多次修改，无法识别。某些服务器上文件的时间戳可能会被无意中更改。
  - b. **ETag / If-None-Match :**
    - **首次响应:** 服务器在响应头中加入 `ETag`，表示资源内容的唯一标识符（类似文件指纹，如 Hash 值）。
    - **后续请求:** 浏览器在请求头中加入 `If-None-Match`，值为上次收到的 `ETag` 的值。
    - **服务器验证:** 服务器比较 `If-None-Match` 的值和资源当前的 `ETag` 值。
    - **优点:** 比 `Last-Modified` 更精确，能识别出内容未变但时间戳变动的情况。优先级高于 `Last-Modified`。

## 场景应用:

- **强缓存:**
  - **适用场景:** 对于**长期不会改变**的静态资源，如使用文件名 Hash 的 JS/CSS/图片文件 (`main.a1b2c3d4.js`)。可以设置一个很长的 `max-age` (例如一年)，实现最优的加载性能。
- **协商缓存:**
  - **适用场景:** 对于**可能会改变，但又希望尽量利用缓存**的资源，如 `index.html`。HTML 文件通常不能设置长缓存，因为它需要及时反映应用的更新。使用协商缓存可以让浏览器每次都验证一下 HTML 是否是最新版，如果是，则返回 304，节省了下载文件本身的带宽。

---

## 6. 首屏加载优化的具体方法

首屏加载优化是性能优化的重中之重，直接关系到用户留存率。



- **Code Splitting (代码分割):**

- **原理:** 将整个应用的 JavaScript 代码库，从一个巨大的 bundle 文件，拆分成多个小的 chunk 文件。然后根据路由、组件或其他逻辑按需加载。
- **实现:**
  - **Webpack:** 使用动态导入 `import()` 语法，Webpack 会自动将动态导入的模块分割成一个独立的 chunk。
  - **React:** 使用 `React.lazy` 和 `Suspense`。
  - **Vue:** 使用路由懒加载。
- **效果:** 大幅减小了首屏需要加载的 JS 体积，只加载当前页面必需的代码，从而加快 FCP 和 TTI。

- **Tree Shaking (摇树优化):**

- **原理:** 在打包过程中，静态分析代码的 `import` / `export` 语句，找出哪些导出的模块没有被实际使用到，然后将这部分“死代码” (dead code) 从最终的 bundle 中移除。
- **前提:** 必须依赖于 **ES6 Module (ESM)** 的静态结构 (`import` / `export`)。CommonJS 的 `require` 是动态的，无法进行有效的 Tree Shaking。
- **效果:** 进一步减小了 JS bundle 的体积，特别是在引入像 `lodash-es` 这样的大型工具库时，只打包用到的函数，效果非常显著。

- **图片懒加载 (Image Lazy Loading):**

- **原理:** 页面初始化时，不设置 `<img>` 标签的 `src` 属性，而是将图片 URL 放在一个自定义属性中 (如 `data-src`)。通过 JavaScript 监听页面的滚动事件，判断图片是否进入了浏览器的可视区域。如果进入，则将 `data-src` 的值赋给 `src` 属性，触发图片加载。
- **实现:**
  - **传统方式:** 监听 `scroll` 事件，并使用 `getBoundingClientRect()` 计算元素位置。但 `scroll` 事件频繁触发，需要做节流 (throttling) 优化。
  - **现代方式:** 使用 `IntersectionObserver` API。这是一个更高效的 API，专门用于检测元素与视口的交叉状态，性能远好于监听 `scroll`。
  - **原生支持:** 现在 `<img loading="lazy">` 属性已被大多数现代浏览器支持，可以实现原生的图片懒加载。
- **效果:** 减少了首屏时不必要的图片请求，加快了页面的 `onload` 时间，节省了用户流量。

---

## 7. 前端安全问题: XSS 与 CSRF

前端安全是每个前端工程师的必修课。

## XSS (Cross-Site Scripting, 跨站脚本攻击):

- **攻击原理:** 攻击者向 Web 页面中注入恶意的脚本 (JavaScript, HTML, CSS 等), 当其他用户浏览该页面时, 这些恶意脚本会在用户的浏览器中执行, 从而窃取用户的 Cookie、Session、LocalStorage, 或者冒充用户进行一些恶意操作。
- **类型:**
  - a. **存储型 XSS:** 恶意脚本被**存储**在服务器的数据库中 (如文章评论、用户个人信息)。当用户请求包含这些数据的页面时, 脚本被执行。危害最大, 影响范围最广。
  - b. **反射型 XSS:** 恶意脚本作为 URL 的一部分 (如 `http://example.com?q=<script>alert(1)</script>`)。当用户点击这个恶意链接时, 服务器从 URL 中获取参数并直接渲染到页面上, 导致脚本执行。通常需要诱骗用户点击。
  - c. **DOM 型 XSS:** 攻击不经过服务器, 而是通过修改前端 DOM 来实现。例如, 前端 JS 代码从 URL 中获取参数并直接 `innerHTML` 到页面上, 攻击者可以构造恶意 URL 来注入脚本。
- **防御措施:**
  - a. **输入过滤:** 对用户输入进行严格的校验和过滤 (但这通常是辅助手段)。
  - b. **输出转义 (核心):** 在将数据渲染到页面时, 对特殊字符进行 HTML 转义。例如, 将 `<` 转义为 `&lt;`, `>` 转义为 `&gt;`。现代前端框架 (如 Vue, React) 默认都会进行转义, 所以要**警惕**使用 `v-html` / `dangerouslySetInnerHTML`。
  - c. **Content Security Policy (CSP):** 在服务器端设置 `Content-Security-Policy` HTTP 头部, 可以限制页面只加载来自可信来源的脚本和资源, 有效防止 XSS 攻击。
  - d. **HttpOnly Cookie:** 设置 Cookie 的 `HttpOnly` 属性, 这样通过 `document.cookie` 就无法读取到 Cookie, 可以防止 XSS 攻击者窃取 Cookie。

## CSRF (Cross-Site Request Forgery, 跨站请求伪造):

- **攻击原理:** 攻击者诱导已登录的用户, 在用户不知情的情况下, 点击一个恶意链接或访问一个恶意页面, 这个页面会向被攻击的网站发送一个伪造的请求 (如转账、修改密码)。由于这个请求是**从用户自己的浏览器发出的, 并且会自动携带用户的 Cookie**, 因此服务器会认为是用户的合法操作。
  - **核心:** 攻击者利用了浏览器自动发送 Cookie 的机制, 但攻击者本身拿不到 Cookie。
- **防御措施:**
  - a. **验证 Referer:** 检查 HTTP 请求头中的 `Referer` 字段, 确保请求来自合法的源站点。但 `Referer` 可以被伪造。
  - b. **CSRF Token (常用):**
    - 用户登录后, 服务器生成一个随机的、唯一的 Token, 并将其存储在 Session 中, 同时下发给前端 (可以放在 `meta` 标签或 cookie 中)。

- 前端在发送所有关键请求（特别是 POST, PUT, DELETE）时，都需要在请求参数或请求头中带上这个 Token。
- 服务器收到请求后，会比较请求中的 Token 和 Session 中存储的 Token 是否一致。由于攻击者无法获取到这个 Token，所以伪造的请求就会被拒绝。

### c. SameSite Cookie 属性:

- 设置 Cookie 的 `SameSite` 属性为 `Strict` 或 `Lax`。
  - `SameSite=Strict`: 完全禁止第三方 Cookie，浏览器在任何跨站请求中都不会携带 Cookie。
  - `SameSite=Lax`: 在大多数跨站请求中不发送 Cookie，但在一些顶层导航（如点击链接）时会发送。
  - 这是目前防御 CSRF 的一个非常有效且简单的手段。
- 

## 8. 在线协作文档编辑器的技术选型与架构设计

这是一个非常复杂且有挑战性的项目，需要综合考虑多个技术难点。

### 技术选型:

- **前端框架:** React 或 Vue。React 的生态和对复杂状态管理的解决方案（如 Redux, MobX）更成熟，可能更适合这种复杂应用。
- **编辑器核心:**
  - **自研:** 基于 `contenteditable` 或完全自绘 (Canvas)。`contenteditable` 坑很多，但生态成熟，可以基于 Prosemirror 或 Slate.js 这样的富文本编辑器框架来构建，它们提供了优秀的数据模型和插件化架构。
  - **Canvas 渲染:** 性能更好，可以实现像素级的精准控制（像 Figma），但开发成本极高，需要自己实现光标、选区、排版、事件处理等所有功能。
- **状态管理:** Redux 或 MobX。需要一个集中、可预测的状态容器来管理复杂的文档数据和 UI 状态。
- **通信:** WebSocket。实现客户端与服务器之间的低延迟、双向实时通信。
- **数据模型:** 需要一个清晰的数据结构来表示文档内容，比如用一个 JSON 结构（类似 Slate.js 的 model）来描述段落、文本、样式等。

### 架构设计:

#### 1. 分层架构:

- **视图层 (View):** 负责渲染文档内容，处理用户交互。
- **逻辑层 (Logic/Controller):** 处理业务逻辑，如用户操作的意图转换、与协同模块的交互。

- **数据模型层 (Model):** 维护文档的当前状态。
- **协同引擎层 (Collaboration Engine):** 负责处理多人协同的核心逻辑。
- **通信层 (Communication):** 封装 WebSocket 连接、消息收发、心跳检测、断线重连。

## 2. 核心技术难点及解决方案:

- **多人实时协同与冲突处理:** 这是最核心的难点。
  - **问题:** 当多个用户同时编辑文档的同一部分时, 如何保证最终结果的一致性?
  - **解决方案:**
    - **OT (Operational Transformation, 操作转换):** 业界成熟的方案 (Google Docs 使用)。核心思想是: 当一个客户端的操作 (Operation) 到达服务器时, 如果在此期间服务器已经接收了其他客户端的操作, 那么需要对这个操作进行“转换”, 使其在新的文档状态上能正确应用。OT 算法非常复杂, 实现难度高。
    - **CRDT (Conflict-free Replicated Data Types, 无冲突复制数据类型):** 一种更现代的、在分布式系统中流行的数据结构。CRDT 的设计保证了即使操作以不同顺序应用, 只要所有操作都应用了, 最终各个副本的状态总会收敛到一致。它天然避免了冲突, 实现起来比 OT 相对简单, 非常适合 P2P 或离线编辑的场景。
- **大数据量渲染性能:**
  - **问题:** 一个几十上百页的文档包含成千上万的 DOM 节点, 一次性全部渲染会导致页面卡顿、崩溃。
  - **解决方案:**
    - **虚拟滚动/虚拟列表 (Virtualization):** 只渲染用户当前视口内可见的部分文档内容, 以及视口上下方的一小部分缓冲区。当用户滚动时, 动态更新渲染的节点。这是解决大数据渲染的通用方案。
- **状态历史与撤销/重做 (Undo/Redo):**
  - **问题:** 需要实现可靠的撤销/重做功能, 并且在协同环境中不出错。
  - **解决方案:** 不能简单地保存文档快照 (开销太大)。应该维护一个操作历史栈 (Operation Stack)。每次用户操作都记录为一个可逆的操作。撤销就是应用这个操作的逆操作, 重做就是重新应用这个操作。在协同环境中, 历史栈的管理需要与 OT/CRDT 算法结合, 确保撤销/重做不会破坏数据一致性。
- **高可用与稳定性:**
  - **问题:** 如何保证 WebSocket 服务的稳定性和可扩展性? 如何处理网络中断?
  - **解决方案:**
    - **后端架构:** 使用支持高并发的后端语言 (如 Go, Elixir), 并设计可水平扩展的分布式 WebSocket 服务。

- **前端机制:** 实现完善的心跳检测和自动断线重连机制。在离线状态下，将用户的操作暂存到本地（如 IndexedDB），待网络恢复后，再发送给服务器进行合并。
- 

### 3. 团队的 CI/CD 流程

我们团队采用了一套基于 GitLab CI/CD 的自动化流程。

#### 流程环节:

##### 1. 本地开发 (Local):

- 开发者在本地 feature 分支上进行开发。
- 使用 Git Hooks 在 `pre-commit` 阶段自动运行 **ESLint** (代码规范检查) 和 **Prettier** (代码格式化)，保证提交的代码风格统一且符合基本规范。

##### 2. 代码提交 (Commit & Push):

- 开发者将 feature 分支推送到 GitLab 远程仓库。

##### 3. 合并请求 (Merge Request):

- 开发者创建一个从 feature 分支到 develop (或 main) 分支的 Merge Request (MR)。
- **CI (持续集成) 触发:** MR 的创建会自动触发 CI Pipeline。
  - **Install:** 安装项目依赖。
  - **Lint & Test:** 再次运行 ESLint 检查，并执行**单元测试 (Unit Tests)** 和**端到端测试 (E2E Tests)**。
  - **Build:** 执行生产环境构建，确保代码可以成功打包。
  - **Preview (可选):** 自动部署到一个临时的预览环境中，方便产品经理和测试人员提前体验。
- **Code Review:** 团队其他成员对 MR 进行代码审查，提出修改意见。

##### 4. 合并与部署 (Merge & Deploy):

- CI Pipeline 通过且 Code Review 通过后，MR 被合并到 develop/main 分支。
- **CD (持续部署) 触发:** 合并到特定分支（如 main 或 release 分支）会自动触发 CD Pipeline。
  - **Build & Dockerize:** 构建生产版本的静态文件，并将其打包成一个 Docker 镜像。
  - **Push to Registry:** 将 Docker 镜像推送到镜像仓库（如 Harbor）。
  - **Deploy:** 触发部署脚本（使用 Kubernetes 或 Ansible），在预发布/生产环境中拉取最新的镜像并部署应用。

##### 5. 上线与监控 (Release & Monitor):

- 部署完成后，进行线上回归测试。

- 通过前端性能监控和错误监控平台，持续观察线上应用的健康状况。

### 我在其中的角色:

- 作为**开发者**，我遵循整个流程，编写代码、单元测试，创建 MR，并参与团队其他成员的 Code Review。
  - 作为**技术负责人/高级工程师**，我参与了 **CI/CD Pipeline 的设计和维护**。比如，优化 GitLab CI 的 YAML 配置文件，引入更快的构建工具，编写自动化测试脚本，以及搭建和维护前端监控体系。
- 

## 6. Node.js 的熟悉程度与区别

我非常熟悉 Node.js，并在多个项目中使用过它。

### 做过什么:

- **BFF (Backend for Frontend)**: 开发 BFF 层，作为前端与后端微服务之间的中间件。它负责整合、裁剪和适配后端微服务返回的数据，使其更适合前端页面展示，同时处理一些与视图强相关的逻辑，如用户认证、模板渲染等。
- **服务端渲染 (SSR)**: 使用 Next.js 或 Nuxt.js 搭建服务端渲染应用，提升首屏加载速度和 SEO。
- **前端工程化工具**: 开发和维护内部的脚手架 (CLI) 工具，用于快速创建项目、代码生成、自动化部署等。
- **API 服务器**: 开发一些小型的、对性能要求不高的 RESTful API 或 GraphQL API，用于内部系统或小型项目。

### 与浏览器环境下 JavaScript 的核心区别:

|         |                                                                                             |                                                      |
|---------|---------------------------------------------------------------------------------------------|------------------------------------------------------|
| 特性      | Node.js 环境                                                                                  | 浏览器环境                                                |
| 全局对象    | global                                                                                      | window                                               |
| 核心模块    | 拥有 http, fs, path, os 等用于服务端操作的核心模块。                                                        | 没有文件、网络等系统级操作能力。                                     |
| 模块系统    | 主要使用 <b>CommonJS</b> (require/module.exports), 现在也原生支持 <b>ESM</b> (import/export)。          | 传统上没有模块系统 (依赖全局变量和 <script> 顺序), 现在原生支持 <b>ESM</b> 。 |
| DOM/BOM | <b>没有</b> document, navigator, location 等 DOM 和 BOM API。                                    | <b>有</b> 完整的 DOM 和 BOM API, 用于与页面交互。                 |
| 事件循环    | 基于 <b>libuv</b> 实现。除了宏任务和微任务队列, 还有 I/O, check, close 等多个阶段。process.nextTick 的优先级高于 Promise。 | 由浏览器实现。通常只有宏任务和微任务队列。                                |
| 应用场景    | 构建服务器、API、命令行工具、桌面应用 (Electron) 等。                                                          | 构建运行在浏览器中的用户界面和交互。                                   |

## 7. 对 TypeScript 的看法

我的看法是：**TypeScript 是现代中大型前端项目开发的标配，它带来的收益远大于成本。**

它解决了什么问题？

- 静态类型检查 (核心):** 能够在**编译阶段**（编码时）就发现大量的类型错误，而不是等到运行时才暴露 Bug，极大地提升了代码的健壮性和可靠性。
- 代码可读性与可维护性:** 类型定义本身就是一种**文档**。通过函数签名、接口定义，我们可以快速理解一个函数需要什么参数、返回什么样的数据，而无需去阅读其内部实现。这在团队协作和长期项目维护中至关重要。



3. **更好的开发体验:** IDE（如 VSCode）可以利用 TypeScript 的类型信息提供**强大的代码补全、智能提示、接口定义跳转和自动重构**功能，开发效率大大提高。
4. **促进更严谨的工程实践:** 它迫使开发者在编码前就思考好数据结构和 API 设计，有助于写出更清晰、更模块化的代码。

### 带来了哪些新的问题？

1. **学习成本:** 需要学习 TypeScript 的语法，如接口 (interface)、泛型 (generic)、枚举 (enum) 等，对于新手有一定门槛。
2. **开发成本:** 需要编写额外的类型定义代码，短期内会增加一些工作量。
3. **类型体操 (Type Gymnastics):** 对于一些复杂的场景，可能需要编写非常复杂的类型定义（条件类型、映射类型等），这被称为“类型体操”，既难写又难懂。
4. **生态集成问题:** 虽然主流库的支持都很好，但在使用一些冷门或老旧的第三方库时，可能需要自己编写类型声明文件 ( `.d.ts` )。

### 项目中类型定义的管理：

1. **接口定义:** 对于后端 API 返回的数据结构，我们会创建一个专门的 `types` 或 `interfaces` 目录，将所有的数据模型接口（如 `User`，`Product`）集中管理，并在需要的地方导入使用。
  2. **代码共享:** 如果是全栈项目，我们会使用 monorepo 架构，将前端和 Node.js 后端共享的类型定义放到一个公共的 `packages/types` 中。
  3. **第三方库:** 优先选择自带类型定义的库。对于没有的，从 `@types/*` 社区仓库安装。如果社区也没有，则在项目中创建一个 `declarations.d.ts` 文件，为其提供基本的类型声明。
  4. **利用工具:** 使用工具如 `json-to-ts` 可以快速根据 API 返回的 JSON 生成 TypeScript 接口，提高效率。
- 

### 5. 如何保证代码质量

保证代码质量是一个多维度、贯穿整个开发生命周期的过程。

除了 **Code Review** 和 **单元测试**，我们还采用了以下方法：

#### 1. 静态代码分析 (Static Code Analysis):

- **Linting:** 使用 **ESLint** 配合各种插件（如 `eslint-plugin-vue`，`eslint-plugin-react-hooks`，`typescript-eslint`）来强制执行团队统一的代码风格和最佳实践，从源头上避免低级错误和坏味道代码。
- **代码复杂度分析:** 使用 SonarQube 或类似工具，定期扫描代码库，分析代码的圈复杂度、重复率、潜在 Bug 等，并将这些指标作为 MR 的质量门禁。

## 2. 编码规范与设计模式:

- **建立团队编码规范:** 制定一份详细的编码规范文档，并要求所有成员遵守。
- **推广设计模式:** 鼓励团队成员学习并应用常见的设计模式（如工厂模式、观察者模式、策略模式），编写出更灵活、可扩展的代码。

## 3. 文档驱动开发 (Documentation-Driven Development):

- 在开始一个复杂模块或组件的开发前，先编写其设计文档和 API 文档。这个过程有助于理清思路，提前发现设计上的问题，也为后续的 Code Review 提供了上下文。

## 4. 测试策略的完善:

- **测试金字塔:** 不仅仅是单元测试，我们还建立了包括**集成测试**（测试多个模块协同工作）和**端到端测试**（使用 Cypress 或 Playwright 模拟真实用户操作）在内的多层次测试体系。
- **测试覆盖率:** 将测试覆盖率作为 CI 中的一个检查项，确保核心逻辑的覆盖率不低于一个阈值。

## 5. 定期的技术分享与重构:

- **技术分享会:** 定期举办团队内部的技术分享，交流最佳实践，学习新的技术，统一技术认知。
- **重构日 (Refactoring Day):** 安排专门的时间来重构代码库中的“技术债”，持续优化代码结构，避免系统腐化。